

Теоретическая информатика*

(вводный курс для учащихся старших классов школы)

Александр Охотин[†]

12 января 2017 г.

Содержание

1	Вводная лекция	2
1.1	Быстрое умножение, алгоритм Карацубы	2
1.2	Символьные строки	3
2	Вычислимость	3
2.1	Машина Тьюринга	3
2.2	Неразрешимые задачи	6
3	Вычислимость с конечной памятью, регулярные языки	8
3.1	Формальные языки	8
3.2	Детерминированные конечные автоматы (DFA)	9
3.3	Ограничения конечных автоматов	10
3.4	Недетерминированные конечные автоматы (NFA)	10
3.5	Построение подмножеств (перевод NFA в DFA)	12
3.6	Регулярные выражения	14
4	Формальные грамматики	15
4.1	Синтаксис языков	15
4.2	Граматики для естественных языков	15
4.3	Граматики для языков программирования	16
4.4	Граматики для языков представления данных	18
4.5	Граматики для биологических данных	18
4.6	Представление абстрактных языков	19
4.7	Определение грамматик	19
4.8	Определение через логический вывод	21
4.9	Определение через перезапись строк	22
5	Алгоритмы и их сложность	22
5.1	Понятие о сложности алгоритма	22
5.2	Задача принадлежности для DFA	23
5.3	Задачи о пустоте и непустоте DFA	23
5.4	Теорема Савича	23
5.5	Теорема Иммермана–Селепчени	24

*Краткое содержание лекций, прочитанных в образовательном центре «Сириус» (Сочи) в январе 2017 г.

[†]СПбГУ, <http://users.utu.fi/aleokh/>, alexander.okhotin@utu.fi.

6 Синтаксический анализ и алгоритмы на строках	25
6.1 Удаление пустых правил	26
6.2 Удаление единичных правил	27
6.3 Приведение к нормальному виду	27
6.4 Алгоритм Кокка–Касами–Янгера	27
6.5 Задачи распознавания свойств грамматик	29
6.6 Нахождение наибольшей общей подпоследовательности	29
A Приложение: быстрое умножение матриц	29

1 Вводная лекция

1.1 Быстрое умножение, алгоритм Карацубы

Умножение в столбик двух n -разрядных чисел требует n^2 умножений разряд на разряд и $O(n)$ сложений. Можно ли быстрее? Высказывалась гипотеза, что, очевидно, быстрее нельзя, и надо только понять, как такие вещи доказывать... А Карацуба (см. Карацуба и Офман [1962]) показал, что можно!

Чтобы перемножить два двухразрядных числа, по определению, нужно вычислить 4 произведения, а потом сложить их.

$$\begin{array}{r}
 \begin{array}{cc}
 a_1 & a_0 \\
 b_1 & b_0
 \end{array} \\
 \hline
 \begin{array}{cc}
 a_1 b_0 & a_0 b_0
 \end{array} \\
 \begin{array}{cc}
 a_1 b_1 & a_0 b_1
 \end{array} \\
 \hline
 \begin{array}{ccc}
 a_1 b_1 & a_1 b_0 + a_0 b_1 & a_0 b_0
 \end{array}
 \end{array}$$

Ещё нужно вычислять перенос, но это просто.



Рис. 1: Анатолий Карацуба (1937–2008).

Первая идея метода Карацубы: два двухразрядных числа можно умножить не за 4, а за 3 умножения. Сперва произведения $a_0 b_0$ и $a_1 b_1$ вычисляются как обычно, а затем перемножаются — внезапно — числа $a_1 + a_0$ и $b_1 + b_0$. Выходит $(a_1 + a_0)(b_1 + b_0) = a_1 b_1 + a_0 b_1 + a_1 b_0 + a_0 b_0$, откуда можно вычесть имеющиеся произведения $a_0 b_0$ и $a_1 b_1$, получив желаемую сумму $a_1 b_0 + a_0 b_1$.

Вторая идея метода Карацубы: точно так же можно перемножать два n -разрядных числа. Они представляются в виде $a = a_1 \cdot 2^{\frac{n}{2}} + a_0$, $b = b_1 \cdot 2^{\frac{n}{2}} + b_0$, где $a_0, a_1, b_0, b_1 \in \{0, \dots, 2^{\frac{n}{2}} - 1\}$ — и дальше всё то же самое, потребуется вычислить 3 произведения чисел длины $\frac{n}{2}$ и $O(n)$ битовых сложений (включая действия с переносом).

Третья идея метода Карацубы: для вычисления 3 произведений чисел вдвое меньшей длины можно рекурсивно использовать тот же самый метод.

Стало быть, глубина рекурсии $\log_2 n$. Подзадача размера n всего одна. Подзадач размера $\frac{n}{2}$ всего 3. Подзадач размера $\frac{n}{2^i}$ всего 3^i . Время работы для каждой подзадачи линейно относительно её размера, и для подзадачи размера $\frac{n}{2^i}$ составляет $O(\frac{n}{2^i})$ шагов. Общее число операций подсчитывается так.

$$\sum_{i=0}^{\log_2 n} 3^i \frac{n}{2^i} = n \sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i = n \frac{\left(\frac{3}{2}\right)^{1+\log_2 n} - 1}{\frac{3}{2} - 1} = O(3^{\log_2 n}) = O(n^{\log_2 3})$$

Основные идеи этого алгоритма впоследствии многократно применялись при построении алгоритмов для решения разнообразных задач. Общий метод получил название **метода «разделяй и властвуй»**. Согласно этому методу, задача разделяется на непересекающиеся подзадачи того же типа, но меньшего размера, каждая подзадача решается отдельно, а потом результаты решения объединяются в решение исходной задачи.

1.2 Символьные строки

Данные представляются *символьными строками*. Сперва задаётся конечное множество символов Σ , называемое *алфавитом*. *Строкой* над алфавитом Σ называется всякая конечная последовательность $w = a_1 \dots a_\ell$, где $\ell \geq 0$, и $a_1, \dots, a_\ell \in \Sigma$ — символы. Число символов в строке называется её *длиной*, $|w| = \ell$. Существует единственная строка длины 0, называемая *пустой строкой* и обозначаемая через ε . Множество всех строк над алфавитом Σ обозначается через Σ^* . Оно счётно для любого алфавита.

Основная операция над строками: *конкатенация*, то есть, приписывание одной строки перед другой. Если $u = a_1 \dots a_m$ и $v = b_1 \dots b_n$, то их конкатенация — это строка $u \cdot v = uv = a_1 \dots a_m b_1 \dots b_n$. Конкатенация считается «умножением» — оно ассоциативно, некоммутативно, имеет «единицу» — нейтральный элемент ε . Соответственно определяются «степени» строки — многократная конкатенация её с самой собою: $w^0 = \varepsilon$, $w^1 = w$, $w^2 = ww$ и т.д.

2 Вычислимость

Предмет вычисления: или вычислить функцию $f: \Sigma^* \rightarrow \Sigma^*$, то есть, по данной на входе строке w вычислить строку $f(w)$, или распознать принадлежность данной на входе строки множеству $L \subseteq \Sigma^*$ и дать ответ «да» или «нет».

Например, если $\Sigma = \{0, 1\}$, а w — двоичная запись числа, то можно рассмотреть функцию, вычисляющую двоичную запись квадрата этого числа, или же задачу распознавания простоты числа, то есть, его принадлежности множеству $L = \{w \mid w \in \{0, 1\}^* \setminus 0\{0, 1\}^*, (w)_2 \text{ — простое}\}$.

2.1 Машина Тьюринга

Математические модели вычисления (1930-е): рекурсивные функции, λ -исчисление, машины Тьюринга; как выяснилось к концу 1930-х, все равносильны друг другу.

Машина Тьюринга: математическая модель вычислений, производимых человеком с помощью карандаша и бумаги, руководствуясь некоторыми правилами.

Тезис Чёрча [1936]: *Всякое эффективно разрешимое свойство описывается рекурсивной функцией.* Тезис Тьюринга [1937]: *Всякое эффективно разрешимое свойство распознаётся машиной Тьюринга.*

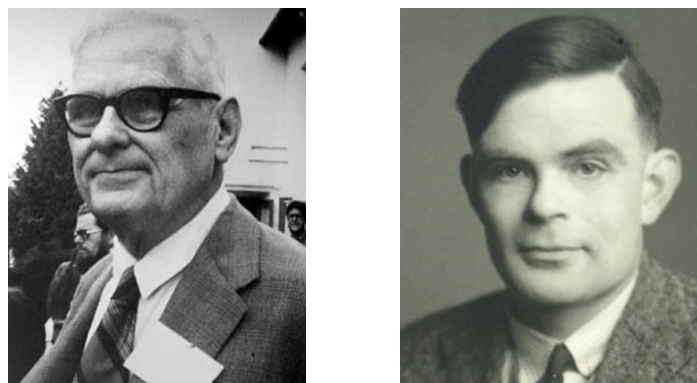


Рис. 2: Алонзо Чёрч (Church) (1903–1995), Алан Тьюринг (Turing) (1912–1954).

В определении машины Тьюринга «бумага» — одномерная, т.е., строка символов произвольной длины, записанная на *ленте*. Лента состоит из *клеток*, и в каждой записывается один символ. Лента полагается неограниченной, т.е., на ней всегда можно найти клетки для записи новых символов, и справа и слева от уже используемых клеток. «Карандаш» становится *головкой*, которая ездит по ленте и в каждый момент времени видит символ в одной клетке. На каждом шаге головка может переписать символ и переместиться на одну клетку вправо или влево. Правила, которыми руководствуется машина, называются её *функцией переходов*.

Определение 1 (Тьюринг [1937]). *Машина Тьюринга — это пятёрка $M = (\Sigma, \Gamma, Q, q_0, \delta)$, компоненты которой имеют следующий вид.*

- Конечное множество Σ — **входной алфавит**.
- Другое конечное множество Γ — **рабочий алфавит**, содержащий все символы, допустимые на ленте, причём $\Sigma \subset \Gamma$, Рабочий алфавит содержит особый символ — пробел: $\sqcup \in \Gamma$, $\sqcup \notin \Sigma$.
- Конечное множество Q — множество **состояний**.
- Есть **начальное состояние** $q_0 \in Q$.
- **Функция переходов** $\delta: Q \times \Gamma \rightarrow (Q \times \Gamma \times \{-1, +1\}) \cup \{Acc, Rej\}$ определяет поведение машины на каждом шаге. Если машина находится в состоянии $q \in Q$ и обозревает символ $a \in \Gamma$, то $\delta(q, a)$ — это тройка (q', a', d) , где $q' \in Q$ — новое состояние, a' — символ, записываемый на ленте вместо a , и $d \in \{-1, +1\}$ направление перемещения головки¹.
- Если машина переходит в **принимаящее состояние** $q_{acc} \in Q$ или в **отвергающее состояние** $q_{rej} \in Q$, то она останавливается.

¹Иногда также разрешают переходы с неподвижной головкой, т.е., $d = 0$. Такие переходы не увеличивают возможности машин и не облегчают их построение.

Лента бесконечна в обе стороны, начальная конфигурация: состояние q_0 , головка смотрит на первый символ входной строки.

Конфигурация машины Тьюринга — это строка вида $\alpha q a \beta$, где $\alpha, \beta \in \Gamma^*$, $a \in \Gamma$ и $q \in Q$, означающая, что машина находится в состоянии q , головка обозревает указанный символ a , и на ленте записаны символы $\alpha a \beta$, окружённые бесконечным числом пробелов в обоих направлениях.

На данной входной строке $w \in \Sigma^*$ машина начинает своё вычисление в **начальной конфигурации** $q_0 w$, то есть, в состоянии q_0 и с символами w на ленте, окружённые пробелами в обоих направлениях ($\dots \sqcup w \sqcup \dots$).

На каждом шаге, если машина находится в конфигурации $\alpha q a \beta$, то её конфигурацию на следующем шаге однозначно определена в соответствии со значением функции переходов для текущего состояния $q \in Q$ и текущего символа $a \in \Gamma$. Если головка едет налево, следующая конфигурация имеет вид $\alpha q' b a' \beta$, и это определяется так.

$$\alpha b q a \beta \vdash \alpha q' b a' \beta, \quad \text{если } \delta(q, a) = (q', a', -1)$$

Если же машина находится у левого края непустого содержимого ленты, то в следующей конфигурации дописывается новый пробел.

$$q a \beta \vdash q' \sqcup a' \beta, \quad \text{если } \delta(q, a) = (q', a', -1)$$

Если машина едет направо, случай самого правого символа также рассматривается отдельно.

$$\begin{aligned} \alpha q a b \beta \vdash \alpha a' q' b \beta, & \quad \text{если } \delta(q, a) = (q', a', +1) \\ \alpha q a \vdash \alpha a' q' \sqcup, & \quad \text{если } \delta(q, a) = (q', a', +1) \end{aligned}$$

Таким образом однозначно определяется конечная или бесконечная последовательность конфигураций, называемая **вычислением** машины на строке w . Вычисление может или **остановиться** на некотором шаге, в том смысле, что машина выполнит команду *Acc* или *Rej*, или же оно может продолжаться бесконечно, в каком случае говорится, что машина **зацикливается**. Строка **принимается** машиной Тьюринга M , если машина останавливается на ней в принимающем состоянии. Множество, распознаваемое машиной — это множество всех строк, которые она принимает.

$$L(M) = \{ w \mid q w \vdash \dots \vdash \alpha q_{acc} a \beta \text{ для некоторых } \alpha, \beta, a \}$$

Функцию переходов $\delta: Q \times \Gamma \rightarrow (Q \times \Gamma \times \{-1, +1\}) \cup \{Acc, Rej\}$ обычно записывают в виде таблицы, где строки соответствуют состояниям, столбцы — символам рабочего алфавита, и в каждой клетке написана или тройка вида (новое состояние, записываемый символ, направление перемещения головки), или команда остановки (*Acc* или *Rej*).

Пример 1. Пусть $M = (\Sigma, \Gamma, Q, q_0, \delta)$ — машина Тьюринга со входным алфавитом $\Sigma = \{a, b\}$, рабочим алфавитом $\Gamma = \{a, b, \sqcup\}$ и множеством состояний $Q = \{q_0, q_1, q_2, q_3\}$, и с функцией переходов, задаваемых следующей таблицей.

	a	b	$\sqcup$
q_0	$(q_1, \sqcup, +1)$	<i>Rej</i>	<i>Acc</i>
q_1	$(q_1, a, +1)$	$(q_1, b, +1)$	$(q_2, \sqcup, -1)$
q_2	<i>Rej</i>	$(q_3, \sqcup, -1)$	<i>Rej</i>
q_3	$(q_3, a, -1)$	$(q_3, b, -1)$	$(q_0, \sqcup, +1)$

Эта машина останавливается на любом входе и принимает строки из множества $\{a^n b^n \mid n \geq 0\}$.

В состоянии q_0 машина стирает первое a входной строки и затем движется направо до конца в состоянии q_1 . Достигнув конца, машина стирает последнее b в состоянии q_2 и возвращается в начало строки в состоянии q_3 . Тогда машина переходит в состояние q_0 и продолжает делать то же самое, пока или все символы входной строки будут стёрты (в этом случае машина принимает), или же будет получена строка неправильного вида, то есть, начинающаяся с b или заканчивающаяся на a . Последнее рано или поздно произойдёт для всякой входной строки не вида $a^n b^n$.

Примеры вычислений данной машины приведены в таблице 1.

В данном примере, как и в большинстве примеров, машина работает достаточно медленно — за время $O(n^2)$. Однако можно доказать, что любой язык программирования высокого уровня можно скомпилировать в машину Тьюринга, причём не более чем с полиномиальным замедлением времени работы по сравнению с тем, как работает обычная программа. Это показывает, что машина Тьюринга и в наши дни остаётся правильной математической моделью вычисления.

2.2 Неразрешимые задачи

Полезность машины Тьюринга в том, что для неё можно математически доказать вещи, доказательство которых на таких моделях вычисления, как, скажем, язык C++, будут слишком трудоёмкими².

Существует множество строк, не распознаваемое никакой машиной Тьюринга.

Пусть $M = (\Sigma, \Gamma, Q, q_0, \delta)$ — машина Тьюринга. Её можно записать математическими значками на бумаге, как в примере 1, и ничего не мешает записать её и в виде символьной строки — как любая информация может быть записана в виде символьной строки.

Описание машины Тьюринга M над алфавитом $\{a, b\}$ — это строка $\sigma(M) \in \{0, 1\}^*$, определяемая следующим образом. Пусть множество состояний — $Q = \{q_1, \dots, q_n\}$ (если состояния назывались как-то иначе, их можно переименовать без ущерба для распознаваемого множества). Пусть входной алфавит — $\Sigma = \{a_1, \dots, a_\ell\}$, рабочий алфавит — $\Gamma = \{a_1, \dots, a_\ell, a_{\ell+1}, \dots, a_m\}$, последний символ — пробел, $a_m = \sqcup$; начальное состояние — q_1 . Тогда машина кодируется так, где символ произведения означает несколько строк, записанных одна за одной.

$$\sigma(M) = 1^\ell 01^m 01^n 0 \left(\prod_{\delta(q_i, a_j) = (q_k, a_r, d)} 1^i 01^j 01^k 01^r 01^{d+1} 0 \right) 0 \left(\prod_{\delta(q_i, a_j) = Acc} 1^i 01^j 0 \right)$$

Далее, всякой машине Тьюринга со входным алфавитом $\Sigma = \{0, 1\}$ можно дать на входе её собственное описание $\sigma(M)$. Машина может принять своё собственное описание или не принять. Соответственно, определяются множества машин, принимающих свои собственные описания (L_1) и не принимающих (L_0).

$$\begin{aligned} L_1 &= \{ \sigma(M) \mid \sigma(M) \in L(M) \} \\ L_0 &= \{ \sigma(M) \mid \sigma(M) \notin L(M) \} \end{aligned}$$

Теорема 1. Множество L_0 не распознаётся никакой машиной Тьюринга.

² Действительно, всякое доказательство, говорящее о «любой программе» на C++, будет вынуждено включать в себя даже не стандарт языка C++, а целиком компилятор, запрограммированный в виде логических умозаключений — в то время как для машины Тьюринга всё уместится в определение 1.

Таблица 1: Вычисления машины Тьюринга из примера 1 на входных строках $w = aaabbb$, $w' = aaabb$ и $w'' = aabbb$.

Доказательство. Доказательство — по существу, диагональный метод Кантора.

Делается предположение, что существует машина Тьюринга M_0 , распознающая множество $L(M_0) = L_0$. Рассматривается строка $\sigma(M_0)$. Оп определению L_0 , строка $\sigma(M_0)$ принадлежит множеству $L(M_0)$ тогда и только тогда, когда верно $\sigma(M_0) \notin L(M_0)$. Получено противоречие — следовательно, такой машины Тьюринга M_0 не существует. $\square$

Для множества L_1 можно показать, что оно распознаётся машиной Тьюринга, однако эта машина Тьюринга непременно будет заикливаться на некоторых строках. Иными словами, в классе машин Тьюринга, останавливающихся на любой входной строке, множество L_1 также неразрешимо.

Множества L_0 и L_1 интересны тем, что они формализуют задачу анализа программ в её простейшем виде. И оказывается, что для полноценной модели вычисления эта задача всегда будет неразрешима.

3 Вычислимость с конечной памятью, регулярные языки

Конечный автомат: модель вычислений с конечной памятью. У каждого конечного автомата — конечное множество из n состояний, то есть, $\log n$ битов памяти, и с помощью этих состояний он должен распознавать входные данные неограниченной длины.

3.1 Формальные языки

В теоретической информатике множества строк часто называются «формальными языками» или просто «языками». Если Σ — алфавит, то Σ^* — множество всех строк над ним, и языком называется всякое множество $L \subseteq \Sigma^*$.

Основные действия над языками. Пусть $K, L \subseteq \Sigma^*$. Так как языки — это множества, для них есть обычные булевы действия над множествами.

$$\begin{aligned} K \cup L &= \{ w \mid w \in K \text{ или } w \in L \} && (\text{объединение } K \text{ и } L) \\ K \cap L &= \{ w \mid w \in K \text{ и } w \in L \} && (\text{пересечение } K \text{ и } L) \\ K \setminus L &= \{ w \mid w \in K \text{ и } w \notin L \} && (\text{вычитание } L \text{ из } K) \end{aligned}$$

Далее, определяется поэлементная конкатенация двух языков.

$$K \cdot L = \{ uv \mid u \in K \text{ и } v \in L \} \quad (\text{конкатенация } K \text{ и } L)$$

Для конкатенации можно определить её степени.

$$L^k = \underbrace{L \cdot \dots \cdot L}_k = \{ w_1 \dots w_k \mid w_1, \dots, w_k \in L \}$$

В частности, $L^0 = \{\varepsilon\}$. Следующее действие — повторение 0 и более раз — называется «звёздочкой Клини».

$$L^* = \bigcup_{k=0}^{\infty} L^k = \{ w_1 \dots w_k \mid k \geq 0, w_1, \dots, w_k \in L \}.$$

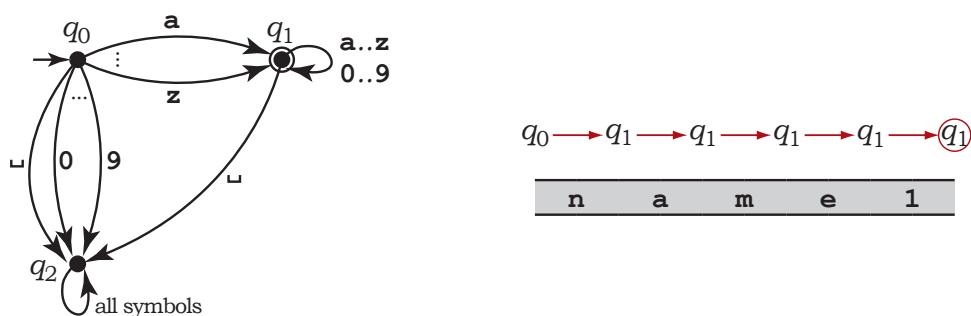


Рис. 3: DFA, распознающий идентификаторы (пример 2), и его вычисление на строке $w = \text{name1}$.

3.2 Детерминированные конечные автоматы (DFA)

Конечный автомат читает входную строку посимвольно слева направо, не возвращаясь назад. Закончив читать строку, автомат выдаёт ответ «да» или «нет».

Понятие *состояния*: конфигурация памяти всего автомата в некоторый момент. Можно сказать, что у автомата с 32 состояниями — 5 битов памяти; однако никакой битовой структуры на самом деле нет, на каждом шаге можно перейти из любого состояния памяти в любое. Число состояний конечно, отсюда название.

Конечный автомат начинает работу в определённом состоянии q_0 , головка видит самый левый символ строке. На каждом шаге текущее состояние и текущий символ определяют состояние в следующий момент, и головка переезжает на одну клетку направо. Состояние после чтения последнего, самого правого символа определяет, принимается строка или отвергается.

Удобно представлять в виде диаграмм переходов, таких как на рис. 3(левый).

Пример 2. Конечный автомат, распознающий множество **идентификаторов**, как в примере 6, задан на рис. 3(левый). Он определён над алфавитом $\Sigma = \{a, \dots, z, 0, \dots, 9, _ \}$, где символ пробела в идентификаторах запрещён.



Рис. 4: Стивен Клини (1909–1994).

Определение 2 (Клини [1951]). **Детерминированный конечный автомат** (*deterministic finite automaton, DFA*; мн. число: *automata*) — пятёрка $\mathcal{A} = (\Sigma, Q, q_0, \delta, F)$, со следующим значением компонентов.

- Σ — алфавит (конечное множество).

- Q — конечное множество состояний.
- $q_0 \in Q$ — начальное состояние.
- Функция переходов $\delta: Q \times \Sigma \rightarrow Q$. Если автомат находится в состоянии $q \in Q$ и читает символ $a \in \Sigma$, то его следующее состояние — $\delta(q, a)$.
- Множество **принимающих состояний** $F \subseteq Q$.

Для всякой входной строки $w = a_1 \dots a_\ell$, где $\ell \geq 0$ и $a_1, \dots, a_\ell \in \Sigma$, вычисление — последовательность состояний $p_0, p_1, \dots, p_{\ell-1}, p_\ell$, где $p_0 = q_0$, и всякое следующее состояние p_i , где $i \in \{1, \dots, \ell\}$, однозначно определено как $p_i = \delta(p_{i-1}, a_i)$.

$$p_0 \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_{\ell-1}} p_{\ell-1} \xrightarrow{a_\ell} p_\ell$$

Строка **принимается**, если последнее состояние p_ℓ принадлежит множеству F — иначе **отвергается**.

Язык, **распознаваемый** автоматом, обозначаемый через $L(\mathcal{A})$ — это множество всех строк, которые он принимает.

Дополнительное обозначение: если DFA начинает вычисление в состоянии $q \in Q$ и читает строку $w = a_1 \dots a_\ell$, то его состояние после её прочтения обозначается через $\delta(q, w)$. Формально, определение функции переходов расширяется до $\delta: Q \times \Sigma^* \rightarrow Q$, и определяется как $\delta(q, \varepsilon) = q$ и $\delta(q, aw) = \delta(\delta(q, a), w)$. В этих обозначениях, язык, распознаваемый DFA, кратко определяется так.

$$L(\mathcal{A}) = \{ w \in \Sigma^* \mid \delta(q_0, w) \in F \}$$

3.3 Ограничения конечных автоматов

Пример 3. Язык $L = \{ a^n b^n \mid n \geq 0 \}$ не распознаётся никаким конечным автоматом.

Доказательство. От противного: пусть он распознаётся некоторым DFA $\mathcal{A} = (\{a, b\}, Q, q_0, \delta, F)$. Для всякой строки a^i , где $i \geq 0$, пусть $q_i = \delta(q_0, a^i)$ — состояние DFA по прочтении этой строки.

Пусть $n = |Q|$. Тогда, читая первую половину строки $w = a^n b^n$, автомат пройдёт через последовательность из $n+1$ состояний $p_0, \dots, p_n$, а после, читая вторую половину, ещё через состояния $p_{n+1}, \dots, p_{2n}$. Так как строка принадлежит языку L , последнее состояние должно быть принимающим: $p_{2n} \in F$.

Поскольку всего различных состояний n , в последовательности $p_0, \dots, p_n$ будет пара одинаковых состояний: то есть, существуют числа i и j , где $0 \leq i < j \leq n$, для которых p_i совпадает с p_j . Тогда вычисление на строке $w' = a^{n-(j-i)} b^n$ имеет вид $p_0, \dots, p_i, p_{j+1}, \dots, p_n, \dots, p_{2n}$, то есть, автомат не замечает, что из строки вырезали кусок. Получается, что автомат принимает строку w' , которая не лежит в L — противоречие. $\square$

3.4 Недетерминированные конечные автоматы (NFA)

Недетерминированный конечный автомат (nondeterministic finite automaton, NFA) — обобщение детерминированного автомата способное *делать ряд догадок* в ходе вычисления. В некоторых состояниях и по некоторым символам, у NFA может быть определено несколько возможных переходов, и когда автомат находится в таком положении, он угадывает одно из возможных продолжений своего вычисления. Если существует некоторая последовательность догадок, приводящая к принятию входной строки, то считается, что автомат её принимает; строка отвергается только если она не была бы принята ни для какой последовательности догадок. Иными словами, *догадки автомата всегда правильны*.

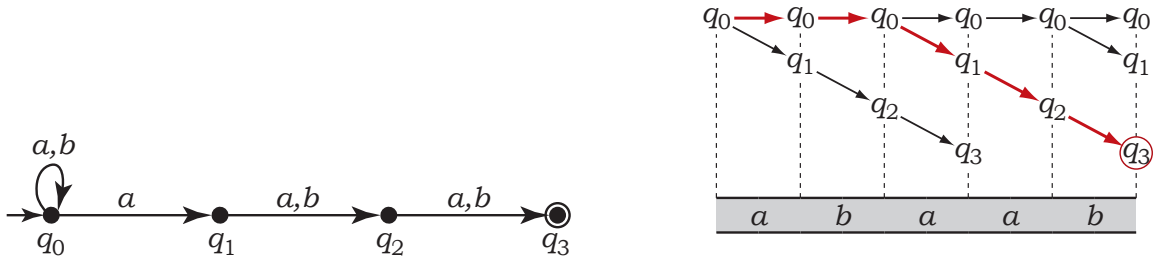


Рис. 5: (левый) NFA из примера 4, угадывающий третий символ с конца; (right) четыре вычисления на строке $w = abaab$, из которых одно — принимающее.

Пример 4. Язык всех строк над алфавитом $\Sigma = \{a, b\}$, в которых третий символ с конца — a , распознаётся NFA на рис. 5(левый).

Его вычисления на строке $abaab$ представлены на рис. 5(правый); в их числе — одно принимающее вычисление.

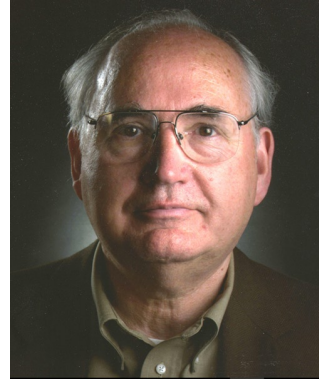


Рис. 6: Майкл Рабин (род. 1931) и Дана Скотт (род. 1932).

Определение 3 (Рабин и Скотт [1959]). *Недетерминированный конечный автомат (NFA) — это пятёрка $\mathcal{B} = (\Sigma, Q, Q_0, \delta, F)$, со следующим значением компонентов.*

- Σ — алфавит.
- Q — конечное множество состояний.
- $Q_0 \in Q$ — множество начальных состояний (автомат угадывает, в каком из них начать).
- Функция переходов $\delta: Q \times \Sigma \rightarrow 2^Q$ даёт множество возможных следующих состояний. Находясь в состоянии $q \in Q$ и читая символ $a \in \Sigma$, автомат может перейти в любое состояние из $\delta(q, a)$.
- Множество **принимающих состояний** $F \subseteq Q$.

Вычисление на входной строке $w = a_1 \dots a_\ell$ — это всякая последовательность состояний $p_0, p_1, \dots, p_{\ell-1}, p_\ell$, где $p_0 \in Q_0$, и $p_i \in \delta(p_{i-1}, a_i)$ для всех $i \in \{1, \dots, \ell\}$,

$$p_0 \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_{\ell-1}} p_{\ell-1} \xrightarrow{a_\ell} p_\ell$$

Вычисление — **принимающее**, если последнее состояние p_ℓ принадлежит множеству F ; иначе оно называется **отвергающим**.

Язык, распознаваемый автоматом, обозначаемый через $L(\mathcal{B})$ — это множество всех входных строк, на которых есть хотя бы одно принимающее вычисление.

В этих обозначениях. NFA в примере 4 имеет множество состояний $Q = \{q_0, q_1, q_2, q_3\}$, где q_0 — единственное начальное состояние ($Q_0 = \{q_0\}$), q_3 — единственное принимающее ($F = \{q_3\}$), и значения функции переходов δ приведены в следующей таблице.

δ	a	b
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
q_3	$\emptyset$	$\emptyset$

Тогда принимающее вычисление на $w = abaab$ обозначается так.

$$q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3$$

DFA может рассматриваться как особый случай NFA, с единственным начальным состоянием ($|Q_0| = 1$) и с единственным следующим состоянием в каждом переходе ($|\delta(q, a)| = 1$ для всех q и a).

3.5 Построение подмножеств (перевод NFA в DFA)

NFA отличается от DFA тем, что может иметь несколько различных вычислений на одной и той же входной строке. Все эти вычисления проходят те же символы в том же порядке, и отличаются только в состояниях. Можно считать, что все они происходят одновременно. Например, как показано на рис. 5(правом), после чтения первых трёх символов есть возможных вычисления, находящиеся в состояниях q_0 , q_1 и q_3 , соответственно. DFA может вычислить множество этих состояний.

Для NFA на рис. 5(левом), моделирующий его работу DFA, читая ту же самую строку $w = abaab$ пройдёт через следующую последовательность состояний-подмножеств: $\{q_0\}$, $\{q_0, q_1\}$, $\{q_0, q_2\}$, $\{q_0, q_1, q_3\}$, $\{q_0, q_1, q_2\}$, $\{q_0, q_2, q_3\}$. Их нетрудно видеть на рис. 5(правом).

Следующая лемма даёт общее построение.

Лемма 1 («построение подмножеств», Рабин и Скотт [1959]). Пусть $\mathcal{B} = (\Sigma, Q, Q_0, \delta, F)$ — произвольный NFA. Тогда существует DFA $\mathcal{A} = (\Sigma, 2^Q, Q_0, \delta', F')$, состояния которого — подмножества Q , который распознаёт тот же язык, что и $\mathcal{B}$. Его переход в каждом состоянии-подмножестве $s \subseteq Q$ по каждому символу $a \in \Sigma$ ведёт во множество состояний, достижимых по a из некоторого состояния из s .

$$\delta'(s, a) = \bigcup_{q \in s} \delta(q, a)$$

Состояние-подмножество $s \subseteq Q$ — принимающее, если оно содержит хотя бы одно принимающее состояние NFA.

$$F' = \{s \mid s \subseteq Q, s \cap F \neq \emptyset\}$$

Общий вид утверждения: по одному вычислительному устройству строится второе, и поведение построенного устройства выражается через поведение исходного устройства. Доказательства подобных результатов обычно начинаются с утверждения о правильности — подробного математического утверждения, описывающего, что новое устройство делает на каждом шаге, и как это связано с работой исходного устройства. Как правило, именно утверждение о правильности содержит в себе главный смысл построения, а его доказательство бывает неинтересным.

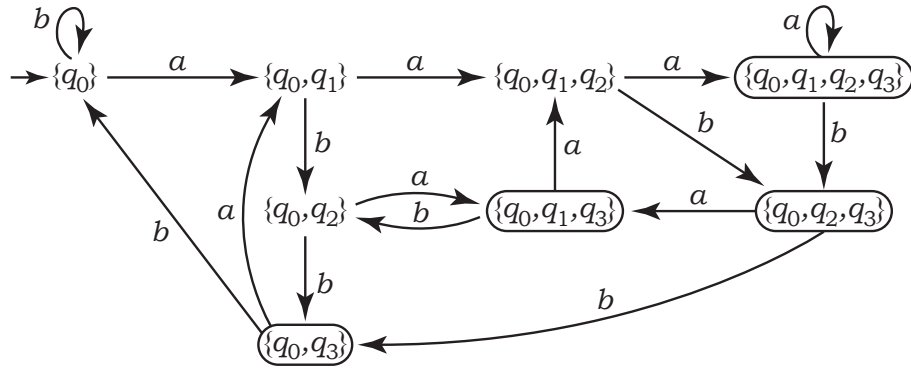


Рис. 7: DFA, моделирующий работу NFA из примера 4, полученный построением подмножеств (все состояния, содержащие q_3 — принимающие).

Доказательство.

Утверждение о правильности. Для всякой строки $w \in \Sigma^*$, состояние-подмножество, достигаемое DFA по прочтении строки w , содержит элемент q тогда и только тогда, когда хотя бы одно из вычислений NFA на w заканчивается в состоянии q .

Доказывается индукцией по длине строки w .

Далее из утверждения о правильности выводится, что построенный DFA принимает строку $w \in \Sigma^*$ тогда и только тогда, когда её принимает исходный NFA. $\square$

Построение переводит NFA из n состояний в DFA из 2^n состояний-подмножеств. На практике, многие из них обычно бывают недостижимы. Поэтому алгоритм будет строить только состояния-подмножества, достижимые из уже построенных, начиная с Q_0 .

Пример 5. Построение подмножеств, применённое к NFA из 4 состояний из примера 4, производит DFA из 8 достижимых состояний, представленный на рис 7.

Можно заметить, каждое состояние-подмножество по существу кодирует три последних прочитанных символа (q_i принадлежит ему если i -й символ с конца — это a).

В худшем случае построение оптимально по числу состояний: О. Б. Лупанов [1963] построил, для всякого n , такой NFA над $\{a, b\}$ из n состояний, что всякий DFA для этого языка должен содержать хотя бы 2^n состояний.



Рис. 8: Олег Лупанов (1932–2006).

3.6 Регулярные выражения

Строятся с помощью трёх операторов: *выбор, конкатенация, повторение*.

Пример 6. В языках программирования, **идентификатор** обычно определяется как всякая непустая последовательность из букв и цифр, начинающаяся с буквы.

$$\underbrace{(a \mid \dots \mid z)}_{\text{любая буква}} \underbrace{(a \mid \dots \mid z \mid 0 \mid \dots \mid 9)}_{\text{любая буква или цифра}}^*$$

Формальное определение: сперва, какой вид может иметь регулярное выражение (его *синтаксис*), затем — определение языка, задаваемого всяким регулярным выражением.

Определение 4 (Клини [1951]). Регулярные выражения над алфавитом Σ определяются так.

- Символ для пустого множества $\emptyset$ — регулярное выражение.
- Всякий символ a , где $a \in \Sigma$ — регулярное выражение.
- Если α и β — регулярные выражения, то тогда $(\alpha \mid \beta)$, $(\alpha\beta)$ и $(\alpha)^*$ — тоже регулярные выражения.

Всякое регулярное выражение α определяет язык над алфавитом Σ , обозначаемый через $L(\alpha)$. Символ для пустого множества определяет пустое множество.

$$L(\emptyset) = \emptyset$$

Всякий символ из Σ обозначает одноэлементное множество, состоящее из односимвольной строки.

$$L(a) = \{a\}$$

Оператор выбора задаёт объединение множеств.

$$L(\alpha \mid \beta) = L(\alpha) \cup L(\beta)$$

Конкатенация регулярных выражений задаёт конкатенацию языков. Оператор итерации задаёт итерацию.

$$L(\alpha\beta) = L(\alpha) \cdot L(\beta)$$

$$L(\alpha^*) = L(\alpha)^*$$

Лишние скобки можно опускать, используя следующий порядок действий: сперва итерация, затем конкатенация, затем выбор. Например, регулярное выражение $(a \mid bc^*)d$ читается как $(a \mid (b(c^*)))d$.

Синтаксис регулярных выражений можно расширить лишними конструкциями: пустая строка (ε), повторение один и более раз (α^+), необязательная конструкция ($[\alpha]$, что означает « α или ничего»). Всё это можно выразить в терминах определения 4. Пустая строка: $\emptyset^* = \{\varepsilon\}$. Повторение один и более раз (α^+) — как $\alpha\alpha^*$. Необязательная конструкция $[\alpha]$ — как $\alpha \mid \varepsilon$, и в конечном счёте как $\alpha \mid \emptyset^*$. Например, $a^+b \mid \varepsilon$ — это сокращённая запись для $aa^*b \mid \emptyset^*$.

Пример 7 (ср. с примером 4). Множество всех строк над алфавитом $\Sigma = \{a, b\}$, в которых третий символ с конца — a , задаётся регулярным выражением $(a \mid b)^*a(a \mid b)(a \mid b)$.

Теорема 2 (Клини [1951]). Язык распознаётся некоторым DFA тогда и только тогда, когда он определяется неким регулярным выражением.

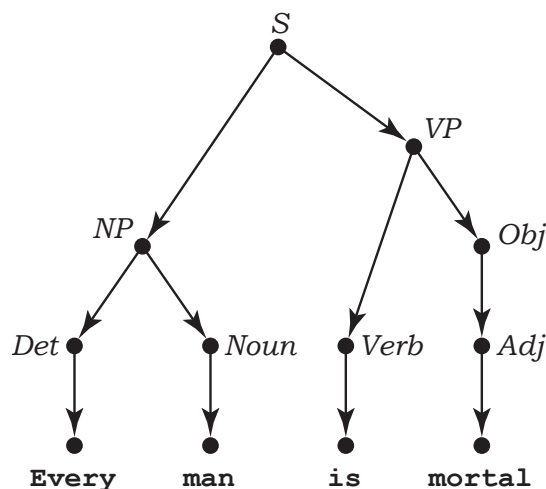


Рис. 9: Дерево разбора предложения *Every man is mortal*.

4 Формальные грамматики

Формальная грамматика — математическая модель синтаксиса языков, как естественных (русский, и т.д.), так и искусственных (C++, XML и т.д.).

4.1 Синтаксис языков

Общие черты синтаксиса языков: (1) представление структурированной информации в виде символьной строки — из речи; (2) не всякая символьная строка — правильно построенное предложение, и синтаксис — это принцип построения правильных предложений; (3) предложения строятся из меньших фрагментов, каждый со своим смыслом, а те в свою очередь из ещё меньших фрагментов, и смысл общего выводится из смысла частей в соответствии с иерархической структурой предложения.

В этом курсе будет изучаться основная разновидность формальных грамматик, в которых фрагменты предложения — это *подстроки* этого предложения, и *правила грамматики* описывают, как несколько подстрок с известными синтаксическими категориями могут быть записаны одна за другой, чтобы получить новую подстроку с определённой синтаксической категорией. Так как никаких других видов грамматик в курсе не будет, такие грамматики будут называться просто *грамматиками*³.

4.2 Грамматики для естественных языков

Разбор предложения по составу: выделяются фразы, словосочетания, строится дерево разбора. В традиционных грамматиках описываются синтаксические категории и то, как они могут сочетаться. Например, английское предложение «**Every man is mortal**» разделяется на группу существительного «**Every man**» (подлежащее) и группу глагола «**is mortal**» (сказуемое). Американские лингвисты, такие как Блумфилд, называли этот подход *анализом непосредственных составляющих* (immediate constituent analysis).

Эти правила можно выразить в виде математического определения грамматически верных предложений, состоящего из высказываний вида «Если вслед за любой группой су-

³К ним прилипло странное название «бесконтекстные грамматики» (context-free; на птичьем языке — «контекстно-свободные»), возникшее в связи тогдашними попытками придумать модель грамматик, где задавались бы правила, применимые в отдельных контекстах. Плоды этих попыток давно и безнадежно устарели, но от них осталось сомнительное название для обыкновенных грамматик, мешающее правильно их понимать уже нескольким поколениям.

ществительного (noun phrase) записать любую группу глагола (verb phrase), то получится предложение». В обозначениях формальных грамматик это правило записывается так.

$$S \rightarrow NP VP$$

В таком виде правила грамматик были впервые записаны Хомским.



Рис. 10: Леонард Блумфилд (1887–1949), Ноам Хомский (род. 1928).

4.3 Грамматики для языков программирования

Алгол 60: первый научно разработанный язык программирования. Грамматики использовались в качестве средства определения синтаксиса. В программистской среде грамматики получили название BNF (Backus–Naur form, форма Бэкуса–Наура), в честь двух членов комитета разработчиков.

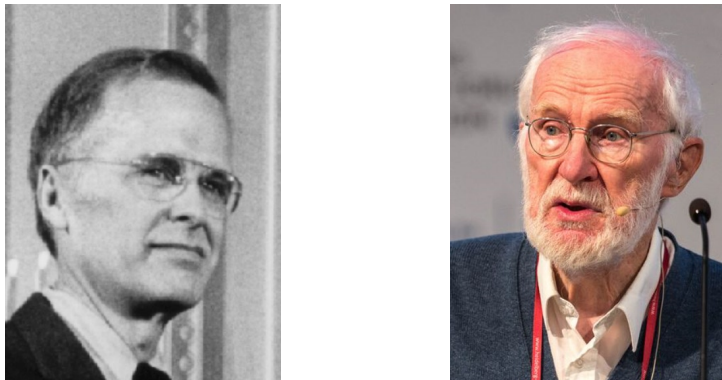


Рис. 11: Джон Бэкус (1924–2007), Петер Наура (1928–2016).

Арифметические выражения.

Пример 8 (Синтаксис арифметических выражений). *Общематематическое определение.*

- *Переменная x — арифметическое выражение.*
- *Число — арифметическое выражение.*
- *Если e и e' — выражения, то $e + e'$ и $e * e'$ — тоже выражения.*
- *Если e — выражение, то (e) — тоже.*

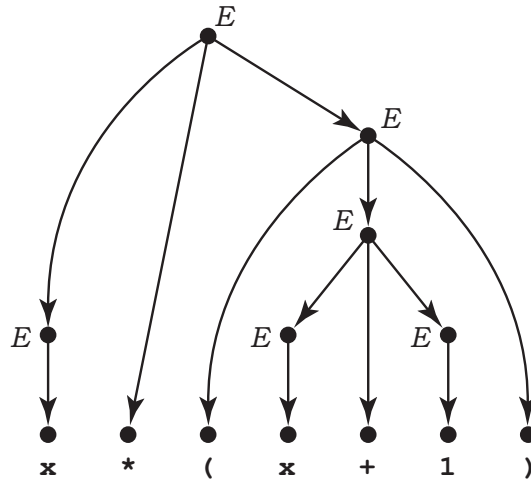


Рис. 12: Дерево разбора выражения $x*(x+1)$ по грамматике из примера 8.

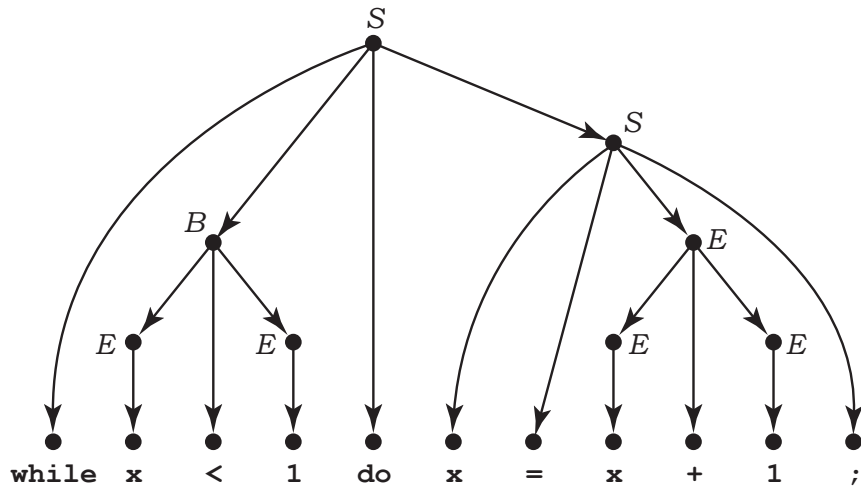


Рис. 13: Дерево разбора оператора `while x<1 do x=x+1;` по грамматике из примера 9.

Ничто другое не является регулярным выражением.

Для простоты, пусть x — переменная, 1 — число. Тогда $x*(x+1)$ — арифметическое выражение, а $x*)1+$ — нет.

В виде формальной грамматики.

$$E \rightarrow x \mid 1 \mid E+E \mid E*E \mid (E)$$

Здесь E — синтаксическая категория арифметических выражений.

Операторы языка программирования `while(x<0) x=x+1;`

Продолжение: логические выражения.

$$B \rightarrow E==E \mid E<E$$

Пример 9.

i. Если x — переменная, а e — арифметическое выражение, то $x = e;$ — оператор.

ii. Если s — оператор и e — логическое выражение, то `while e do s` — тоже оператор.

iii. Если $s_1, \dots, s_k$ — операторы, где $k \geq 0$, то `begin  $s_1 \dots s_k$  end` — оператор.

И т.д., ещё несколько видов операторов.

Пусть S — категория синтаксически правильных операторов.

$$\begin{aligned} S &\rightarrow x = E; \mid \text{while } B \text{ do } S \mid \text{begin } S' \text{ end} \\ S' &\rightarrow SS' \mid \varepsilon \end{aligned}$$

Например, поскольку $x < 1$ — выражение, а $x = x + 1$ — оператор, то `while  $x < 1$  do  $x = x + 1$ ;` — также оператор. Дерево разбора дано на рис. 13.

4.4 Грамматики для языков представления данных

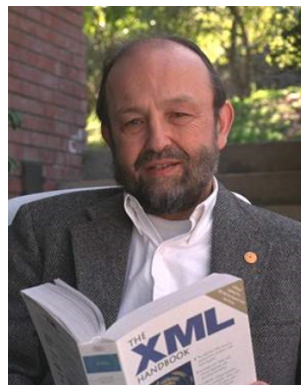


Рис. 14: Чарльз Гольдфарб.

SGML, HTML, XML. Предельно упрощённый синтаксис, удобно для машинной обработки. По существу, это отказ от привычного синтаксиса языков в пользу явного задания дерева разбора: в самой строке записано, как её надо разбирать.

Пример 10. Определение маленького фрагмента HTML, с одной парой тэгов.

- Всякий символ, кроме угловых скобок ($<$, $>$) — правильная HTML-строка.
- Конкатенация двух правильных HTML-строк — тоже правильная HTML-строка.
- Если x — правильная HTML-строка, то `<blockquote>  $x$  </blockquote>` — тоже.

В виде формальной грамматики (символ c обозначает любой знак).

$$S \rightarrow c \mid SS \mid \text{<blockquote> } S \text{ </blockquote>}$$

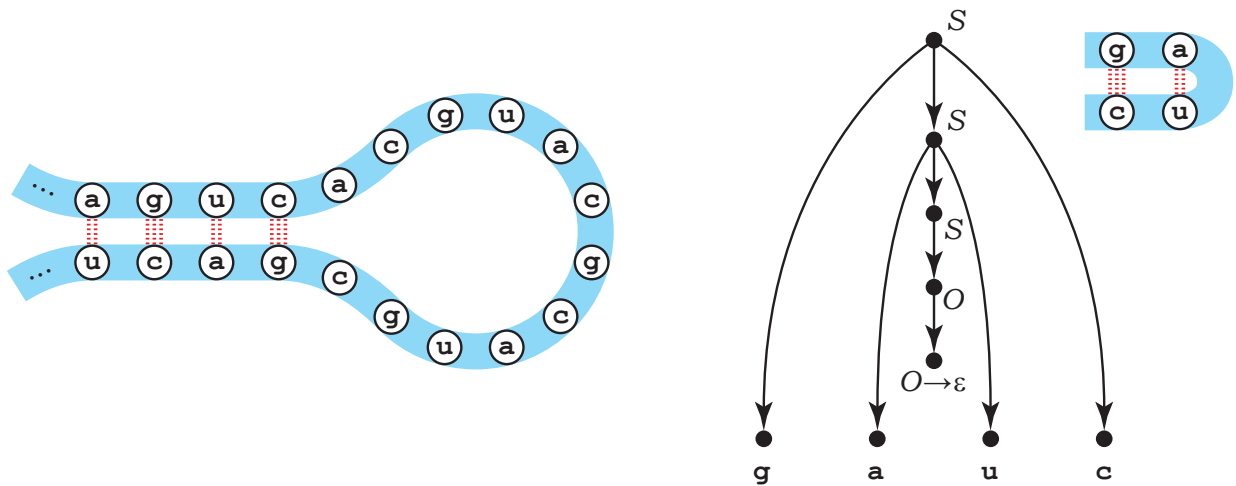
В SGML есть свои грамматики — DTD.

4.5 Грамматики для биологических данных

Первичная структура РНК: последовательность нуклеотидов G (гуанин), C (цитозин), A (аденин), U (урацил). Вторичная структура: последовательность сгибается и соединяется сама с собою, образуя соединения G-C, A-U. Пример вторичной структуры: *шпилька* (hairpin) — как на рис. 15(левый).

Пример 11. Общее определение (с некоторыми погрешностями против биологии).

- Всякая последовательность нуклеотидов — это петля (loop).



- Всякая петля — это шпилька.
- Если последовательность s — это шпилька, то тогда gsc , csg , asu и usa — тоже шпильки.

$$\begin{array}{l} S \rightarrow O \mid gSc \mid cSg \mid aSu \mid uSa \\ O \rightarrow gO \mid aO \mid uO \mid cO \mid \varepsilon \end{array}$$

4.6 Представление абстрактных языков

Множество правильно вложенных последовательностей скобок называется *языком Дика*.

Пример 12. *Общее определение.*

- Пустая строка — правильно вложенная.
- Если u — правильно вложенная, то aub — тоже правильно вложенная.
- Если u и v — правильно вложенные, то их конкатенация uv — тоже.

$$S \rightarrow \varepsilon \mid aSb \mid SS$$

4.7 Определение грамматик

Определение 5. (Формальная) грамматика — это четвёрка $G = (\Sigma, N, R, S)$, состоящая из следующих компонентов.

- Конечное множество **символов** Σ — алфавит определяемого языка.

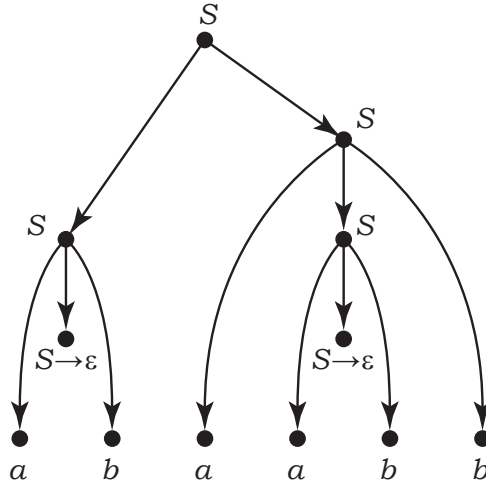


Рис. 16: Дерево разбора строки $abaabb$ по грамматике из примера 12.

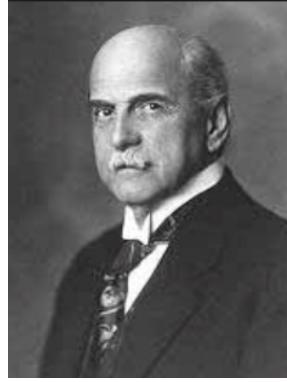


Рис. 17: Вальтер фон Дик (1856–1934).

- Конечное множество N содержит синтаксические категории, определяемые в грамматике. Каждую категорию можно считать свойством, которым всякая строка над алфавитом Σ обладает или не обладает. Обозначаются обычно буквами $A, B, C, \dots$

Чаще всего категории называют «нетерминальными символами» («нетерминалами») — отсюда буква N — поскольку пути в дереве разбора на них не заканчиваются.

- Конечное множество R правил грамматики, каждое из которых описывает возможную структуру строк со свойством $A \in N$ в виде конкатенации $u_0 B_1 u_1 \dots B_\ell u_\ell$, где $B_1, \dots, B_\ell$ ($\ell \geq 0$) — все категории, на которые ссылается правило, а $u_0, u_1, \dots, u_\ell$ — любые символы, написанные между ними.

$$A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell \quad (A \in N, \ell \geq 0, B_1, \dots, B_\ell \in N, u_0, u_1, \dots, u_\ell \in \Sigma^*)$$

- Категория синтаксически правильных предложений $S \in N$, от «sentence». Обычно называют начальным символом.

Общий вид правил часто записывается как $A \rightarrow X_1 \dots X_\ell$, где $X_1, \dots, X_\ell \in \Sigma \cup N$ ($\ell \geq 0$) — все символы и категории, конкатенация которых записана в правиле. Дальнейшее обозначение: $A \rightarrow \alpha$, где $\alpha = X_1 \dots X_\ell$ — строка над алфавитом $\Sigma \cup N$.

Несколько правил $A \rightarrow \alpha_1, \dots, A \rightarrow \alpha_m$ для одной и той же категории $A \in N$ записывают возможную структуру строк с этим свойством. Это такой же оператор выбора, как и в регулярных выражениях, и потому такие правила обычно записываются следующим образом.

$$A \rightarrow \alpha_1 \mid \dots \mid \alpha_m$$

Пример 13. Грамматика для языка Дика записывается как $G = (\Sigma, N, R, S)$, где $\Sigma = \{a, b\}$, $N = \{S\}$ и $R = \{S \rightarrow aSb, S \rightarrow SS, S \rightarrow \varepsilon\}$. Такую грамматику записывают следующим образом.

$$S \rightarrow aSb \mid SS \mid \varepsilon$$

Дерево разбора строки $w = abaabb$ в соответствии с этой грамматикой приведено на рис. 16.

Каждая грамматика $G = (\Sigma, N, R, S)$ задаёт формальный язык над алфавитом Σ , обозначаемый через $L(G) \subseteq \Sigma^*$. Его можно определить как *множество всех строк, для которых есть дерево разбора* — но чтобы сделать это математически строго, потребуется долго определять, что такое дерево разбора. Более удобно использовать приведённые ниже два чисто символических определения.

4.8 Определение через логический вывод

Граматику можно понимать как очень простую логическая систему, в которой выводятся все верные высказывания вида “строка w имеет свойство A ”, обозначаемые через $A(w)$. Например, высказывание $E(x+1)$, выводимое в грамматике из примера 8, означает, что строка $x+1$ является синтаксически правильным арифметическим выражением.

Пример 13(D). В грамматике из примера 13 определена одна категория — категория всех правильно вложенных строк S . Поэтому высказывание вида $S(w)$ можно прочитать как “ w — правильно вложенная строка”.

Утверждение о правильной вложенности строки $w = abaabb$ выводится так.

$$\frac{}{S(\varepsilon)}(S \rightarrow \varepsilon) \quad \frac{S(\varepsilon)}{S(ab)}(S \rightarrow aSb) \quad \frac{S(ab)}{S(aabb)}(S \rightarrow aSb) \quad \frac{S(ab), S(aabb)}{S(abaabb)}(S \rightarrow SS)$$

Этот вывод можно записать в виде дерева доказательства, разделив $S(ab)$ для двух разных подстрок ab .

$$\frac{\frac{S(\varepsilon)}{S(ab)} \quad \frac{S(\varepsilon)}{S(ab)}}{S(abaabb)}$$

Это тот же объект, что и дерево разбора (ср. рис. 16).

Определение 5(D). Для грамматики $G = (\Sigma, N, R, S)$, высказывания имеют вид “строка w имеет свойство A ”, где $w \in \Sigma^*$ и $A \in N$, и обозначаются через $A(w)$.

Пусть $A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell$ — правило, в котором $B_1, \dots, B_\ell \in N$, где $\ell \geq 0$ — это все категории, на которые оно ссылается, а $u_0, u_1, \dots, u_\ell \in \Sigma^*$ — символы между ними. Это правило позволяет сделать следующий логический вывод, для любых строк $v_1, \dots, v_\ell$, где над чертой — посылки, а под чертой — следствие.

$$\frac{B_1(v_1), \dots, B_\ell(v_\ell)}{A(u_0 v_1 u_1 \dots v_\ell u_\ell)}(A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell) \quad (\text{for all } v_1, \dots, v_\ell \in \Sigma^*)$$

Вывод высказывания $A(u)$ — это последовательность таких шагов вывода, где в качестве посылок на каждом шаге используются ранее выведенные высказывания. Если такой вывод существует, это обозначается через $\vdash A(u)$.

Тогда, для всех $A \in N$, определяется $L_G(A) = \{ w \mid \vdash A(w) \}$. Язык, задаваемый грамматикой — $L(G) = L_G(S) = \{ w \mid \vdash S(w) \}$.

4.9 Определение через перезапись строк

Перезапись *сентенциальных форм* — строк над алфавитом $\Sigma \cup N$. Каждая такая строка — это схема предложения, в которой каждое вхождение каждой категории $A \in N$ означает некоторую неуказанную строку со свойством A .

Перезапись начинается с S , что означает *любое правильное предложение*. На каждом шаге перезаписи, некоторая категория A заменяется на правую часть правила для A , получается более определённая сентенциальная форма. Перезапись продолжается, пока не останутся только символы из Σ — то есть, предложение языка.

Пример 13(R). Для грамматики из примера 13, строка $abaabb$ получается перезаписью так.

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abaaSbb \Rightarrow abaabb$$

Определение 5(R) (Хомский). Пусть $G = (\Sigma, N, R, S)$ — грамматика. Если $\eta A \theta$ — строка над совмещённым алфавитом символов и категорий $\Sigma \cup N$ (где $A \in N$ — категория и $\eta, \theta \in (\Sigma \cup N)^*$), и если $A \rightarrow \alpha$ — некоторое правило для A , тогда строка $\eta A \theta$ может быть **перезаписана в $\eta \alpha \theta$ за один шаг**, что обозначается следующим образом.

$$\eta A \theta \Rightarrow \eta \alpha \theta \quad (\text{для всех } A \rightarrow \alpha \in R \text{ and } \eta, \theta \in (\Sigma \cup N)^*)$$

Последовательность строк $\alpha_0, \alpha_1, \dots, \alpha_n$ над алфавитом $\Sigma \cup N$, где $n \geq 0$, называется *цепочкой перезаписи*, если всякая строка α_i может быть перезаписана за один шаг в строку α_{i+1} . Это обозначается так.

$$\alpha_0 \Rightarrow \alpha_1 \Rightarrow \dots \Rightarrow \alpha_n$$

Тогда говорится, что α_0 перезаписывается в α_n за n шагов (обозначение: $\alpha_0 \Rightarrow^n \alpha_n$). Также вводятся обозначения для перезаписи за ноль и более шагов ($\alpha \Rightarrow^* \beta$), за один и более шагов ($\alpha \Rightarrow^+ \beta$), и за не более чем n шагов ($\alpha \Rightarrow^{\leq n} \beta$).

Язык, задаваемый грамматикой.

$$\begin{aligned} L_G(A) &= \{ w \mid w \in \Sigma^*, A \Rightarrow^+ w \} \\ L(G) &= L_G(S) \end{aligned} \quad (A \in N)$$

Определения 5(D) и 5(R) равносильны, то есть, определяют язык $L(G)$ одинаково для любой грамматики G .

5 Алгоритмы и их сложность

5.1 Понятие о сложности алгоритма

Для данной машины Тьюринга M , останавливающейся на любом входе, и для данной входной строки w , время работы M на w , обозначаемое через $t(w)$ — это число шагов в вычислении. Сложность по времени — это функция $t: \mathbb{N} \rightarrow \mathbb{N}$, определяемая как $t(n) = \max_{w \in \Sigma^*, |w|=n} t(w)$

Сложность по памяти определяется на *двуленточной машине Тьюринга*, оснащённой двумя лентами. На первой ленте — входной — даётся входная строка; эта лента доступна только для чтения. Вторая лента — рабочая — изначально пуста, ею можно пользоваться без ограничений. На каждой ленте своя головка. Состояние одно. Функция переходов одновременно видит символы на обеих лентах, на каждом шаге символ на рабочей ленте перезаписывается, и каждая головка едет своей дорогой.

Определение 6. *Двуленточная машина Тьюринга — это пятёрка $M = (\Sigma, \Gamma, Q, q_0, \delta)$, компоненты которой имеют следующий вид.*

- Конечное множество Σ — **входной алфавит**, используемый на
- Другое конечное множество Γ — **рабочий алфавит**, содержащий все символы, допустимые на рабочей ленте. Рабочий алфавит содержит особый символ — пробел: $\sqcup \in \Gamma$, $\sqcup \notin \Sigma$.
- Конечное множество Q — множество **состояний**.
- Есть **начальное состояние** $q_0 \in Q$.
- **Функция переходов** $\delta: Q \times \Sigma \times \Gamma \rightarrow (Q \times \{-1, 0, +1\} \times \Gamma \times \{-1, 0, +1\}) \cup \{Acc, Rej\}$ определяет поведение машины на каждом шаге. Если машина находится в состоянии $q \in Q$ и обзрывает символы $a \in \Sigma$ и $b \in \Gamma$ на двух лентах, то $\delta(q, a, b)$ — это набор вида (q', d_1, b', d_2) , где $q' \in Q$ — новое состояние, b' — символ, записываемый на рабочей ленте вместо b , и $d_i \in \{-1, 0, +1\}$ — направление перемещения головки на i -й ленте.

Объём памяти, требуемый двуленточной машиной Тьюринга на входной строке $w \in \Sigma^*$ — это разность между самым левым и самым правым положением головки на рабочей ленте за время всего вычисления, обозначаемая через $s(w)$. Сложность по памяти — это функция $s: \mathbb{N} \rightarrow \mathbb{N}$, определяемая как $t(n) = \max_{w \in \Sigma^*, |w|=n} s(w)$

5.2 Задача принадлежности для DFA

Дано: DFA A и входная строка w для него. Определить, верно ли, что $w \in L(A)$.

Разрешима в логарифмической памяти.

5.3 Задачи о пустоте и непустоте DFA

Дано: DFA A . Задача о пустоте: определить, верно ли, что $L(A) = \emptyset$. Задача о непустоте: определить, верно ли, что $L(A) \neq \emptyset$.

Если решать на детерминированной машине Тьюринга, то это одна и та же задача: алгоритм для решения одной из них преобразуется в алгоритм для решения другой путём замены ответа «да» на ответ «нет», и наоборот.

Однако если решать на недетерминированной машине Тьюринга, то есть разница.

Задача о непустоте очевидно разрешима в недетерминированной логарифмической памяти: достаточно просто угадать строку, которую принимает полученный на входе DFA.

Это по существу задача о нахождении пути в графе.

5.4 Теорема Савича

Сколько памяти достаточно, чтобы детерминированно решить ту же задачу? Оказывается, что хватит $(\log n)^2$ битов — квадрата объёма памяти, используемого недетерминированным алгоритмом. Известен следующий общий результат.



Рис. 18: Вальтер Савич (Savitch) (род. 1943).

Теорема 3 (Савич [1970]). *Всякая задача, решаемая недетерминированной МТ в памяти $s(n)$, решается детерминированной МТ в памяти $(s(n))^2$.*

Доказательство приводится в частном случае для задачи поиска пути в графе.

Составляется процедура, отвечающая на следующий вопрос: «Верно ли, что из вершины v можно придти в вершину v' не более чем за ℓ шагов?» Обозначение: $P(v, v', \ell)$. Цель: вычислить $P(\text{начальная}, \text{конечная}, n)$.

Вычисляется делением вычисления пополам.

Процедура $P(v, v', t)$

```

1: if  $t = 1$  then
2:   if есть переход из  $v$  в  $v'$  then
3:     «да»
4:   else
5:     «нет»
6: for all  $v''$  — вершина do
7:   if  $P(v, v'', \frac{t}{2}) \wedge P(v'', v', \frac{t}{2})$  then
8:     «да»
9: «нет»

```

Глубина рекурсии — $\log n$. На каждом уровне рекурсии требуется $O(\log n)$ битов памяти, чтобы хранить значения v , v' , t и v'' — всего $O((\log n)^2)$.

В общем случае теоремы Савича вместо вершин графа будут конфигурации машины Тьюринга — но общая идея доказательства такая же.

5.5 Теорема Immerman–Szelepceni

Сколько памяти достаточно, чтобы недетерминированно проверить *несуществование* пути? Оказывается, что столько же — всего лишь $O(\log n)$ битов. Известна следующая общая теорема.

Теорема 4 (Иммерман [1988]; Селепчени [1988]). *Если задача решается недетерминированной МТ в памяти $s(n)$, то её дополнение также решается асимптотически в той же памяти.*

Доказательство — для частного случая: нахождения пути в графе за время $\log n$.

Для всякого числа $i \geq 0$, пусть C_i — множество всех вершин, достижимых из начальной ровно на i шагов. Недетерминированный алгоритм, распознающий несуществование пути, последовательно вычисляет, для всех $i \geq 0$, число $|C_i|$ вершин, достижимых за i шагов. При



Рис. 19: Нейл Иммерман (род. 1953), Роберт Селепчени (Szelepcsényi) (род. 1967).

этом, хоть алгоритм и недетерминированный, он вычисляет это число правильно во всех своих возможных вычислениях.

Это делается **методом индуктивного подсчёта**. За 0 шагов достигается только начальная вершина, то есть, $|C_0| = 1$. Используя число $|C_i|$ вершин, достижимых за i шагов, число $|C_{i+1}|$ вершин, достижимых за $i + 1$ шагов, вычисляется следующим образом.

Рассматриваются все вершин в определённом порядке — так, что каждая из них будет рассмотрена ровно однажды. Рассматривая всякую вершину v , алгоритм недетерминированно угадывает $|C_i|$ различных вершин $v_1, \dots, v_{|C_i|}$, и проверяет, что каждая из них достигается из начальной вершины за i шагов: это делается угадыванием всего пути. Если для какой-то из угаданных вершин достижимость не устанавливается, то всё вычисление алгоритма неудачно. Таким образом, алгоритм проверяет, что все угаданные им вершины действительно принадлежат множеству C_i . Далее, поскольку алгоритм угадывает ровно столько таких вершин, сколько их вообще есть в C_i , он вынужден угадать их все: $\{v_1, \dots, v_{|C_i|}\} = C_i$.

Проверяя каждую вершину v_j , алгоритм смотрит, достижима ли вершина v из v_j за один шаг. Если оказывается, что v достижима хотя бы из одной v_j , то счётчик вершин в C_{i+1} увеличивается на 1. Вычисленное таким образом число $|C_{i+1}|$ не зависит от принятых алгоритмом недетерминированных решений.

Если в какой-то момент устанавливается достижимость конечной вершины, алгоритм возвращает ответ «нет».

Иначе, алгоритм продолжает вычислять значения $|C_i|$ для последовательных значений i , пока он не достигнет i , равному общему числу вершин. Поэтому, если конечная вершина не достигается по путям меньшей длины, то, значит, она вообще не достигается, и потому алгоритм может, оставив сомнения, выдать ответ «да».

6 Синтаксический анализ и алгоритмы на строках

Задача синтаксического анализа: имеется фиксированная грамматика $G = (\Sigma, N, R, S)$. По подаваемой на вход строке $w \in \Sigma^*$ необходимо проверить, является ли она синтаксически правильной относительно грамматики G — т.е., верно ли, что $w \in L(G)$?

Для построения наиболее простого алгоритма, удобно сперва привести грамматику к «нормальному виду».

Построение таблицы.

Время — $O(n^3)$. А можно и быстрее — за время $O(n^\omega)$, где $\omega < 3$ — через умножение матриц и быстрый алгоритм умножения матриц.

Удобно предположить, что грамматика находится в *нормальном иде* Преобразование — в несколько этапов.

6.1 Удаление пустых правил

Цель: по данной грамматике, построить грамматику, задающую тот же язык и не содержащую «пустых правил» вида $A \rightarrow \varepsilon$.

Пример 14. Грамматика, определяющая язык $\{ab, b\}$.

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow \varepsilon \mid a \\ B &\rightarrow b \end{aligned}$$

Правило $A \rightarrow \varepsilon$ хочется убрать. Однако его удаление потребует добавить к правилу $S \rightarrow AB$ дополнительное правило $S \rightarrow B$, соответствующее случаю, когда ε обозначает ε . На выходе такая грамматика.

$$\begin{aligned} S &\rightarrow AB \mid B \\ A &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

Сперва надо узнать, какие категории содержат пустую строку ε . Для грамматики $G = (\Sigma, N, R, S)$ это следующее множество.

$$\text{NULLABLE}(G) = \{ A \mid A \in N, \varepsilon \in L_G(A) \}$$

Algorithm 1 Построение множества $\text{NULLABLE}(G)$.

Грамматика $G = (\Sigma, N, R, S)$, переменная: $W \subseteq N$.

- 1: $W = \emptyset$
- 2: **while** W можно изменить **do**
- 3: $W = \{ A \in N \mid \text{есть правило } A \rightarrow B_1 \dots B_\ell, \text{ где } B_1, \dots, B_\ell \in W \}$

На выходе $W = \text{NULLABLE}(G)$.

Последовательно строятся следующие множества.

$$\text{NULLABLE}_0(G) = \emptyset$$

$$\text{NULLABLE}_{i+1}(G) = \{ A \in N \mid \text{есть правило } A \rightarrow B_1 \dots B_\ell, \text{ где } B_1, \dots, B_\ell \in \text{NULLABLE}_i(G) \}$$

Это последовательность вложенных множеств, рано или поздно множество насыщается.

Лемма 2. Грамматика $G = (\Sigma, N, R, S)$, множество $\text{NULLABLE}(G)$ для неё. Новая грамматика $G' = (\Sigma, N, R', S)$, где R' содержит следующие правила.

$$\begin{aligned} A &\rightarrow X_1 \dots X_\ell && (\text{если в } R \text{ есть правило } A \rightarrow \theta_0 X_1 \theta_1 \dots X_\ell \theta_\ell, \\ &&& \text{где } \ell \geq 1 \text{ и } \theta_0, \dots, \theta_\ell \in \text{NULLABLE}(G)^*). \end{aligned}$$

Тогда, для каждого $A \in N$, $L_{G'}(A) = L_G(A) \setminus \{\varepsilon\}$.

6.2 Удаление единичных правил

Цель: по данной грамматике без пустых правил, построить грамматику, задающую тот же язык и не содержащую «единичных правил» вида $A \rightarrow B$.

Лемма 3. Пусть $G = (\Sigma, N, R, S)$ — грамматика, в которой всякое правило имеет вид $A \rightarrow \alpha$, где $|\alpha| \geq 1$, и, соответственно, $\varepsilon \notin L_G(A)$ для всех $A \in N$. Тогда тот же язык задаётся грамматикой $G' = (\Sigma, N, R', S)$, где R' содержит правило $A \rightarrow \alpha$ тогда и только тогда, когда $\alpha \in (\Sigma \cup N)^* \setminus N$ и есть последовательность правил $A_0 \rightarrow A_1, A_1 \rightarrow A_2, \dots, A_{k-1} \rightarrow A_k, A_k \rightarrow \alpha$ в R , где $A_0 = A$ и $k \geq 0$.

6.3 Приведение к нормальному виду

Определение 7. Грамматика $G = (\Sigma, N, R, S)$ — в нормальном виде Хомского, если всякое правило из R имеет следующий вид.

$$\begin{aligned} A &\rightarrow BC && (B, C \in N) \\ A &\rightarrow a && (a \in \Sigma) \\ S &\rightarrow \varepsilon && (\text{только если } S \text{ не используется в правых частях никаких правил}) \end{aligned}$$

Теорема 5 (Хомский [1959]). Для всякой грамматики можно построить грамматику в н.в.Хомского, задающую тот же язык.

Набросок доказательства. Подготовка: нарезать правила, чтобы длина правой части не превосходила 2.

Шаг 1: удалить все пустые правила вида $A \rightarrow \varepsilon$.

Шаг 2: удалить единичные правила вида $A \rightarrow B$.

Последний шаг: переместить символы из Σ в отдельные правила. □

6.4 Алгоритм Кокка–Касами–Янгера

Синтаксический анализ: для фиксированной грамматики $G = (\Sigma, N, R, S)$ и для данной входной строки $w \in \Sigma^*$, проверить, принадлежит ли строка языку $L(G)$, и если принадлежит, то построить дерево разбора.

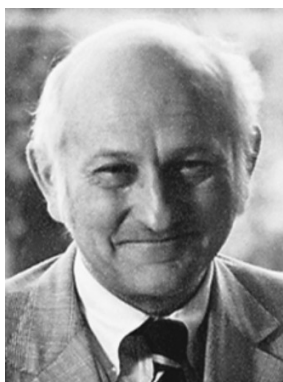


Рис. 20: Джон Кокк (1925–2002), Тадао Касами (1930–2007), Дэниел Янгер.

Грамматика $G = (\Sigma, N, R, S)$ в н.в.Хомского, входная строка $w = a_1 \dots a_n$. Цель: определить для каждой подстроки $a_{i+1} \dots a_j$ и для каждой категории $A \in N$, имеет ли эта

подстрока свойство A . Эти сведения записываются в *таблицу разбора* $T \in (2^N)^{n \times n}$, в которой всякий элемент $T_{i,j}$, где $0 \leq i < j \leq n$ содержит множество категорий для подстроки между позициями $i + 1$ и j .

$$T_{i,j} = \{ A \in N \mid a_{i+1} \dots a_j \in L_G(A) \}$$

Строятся от меньших подстрок к большим. Сперва вычисляются свойства всех подстрок длины 1. Чтобы односимвольная подстрока a_i обладала свойством A , необходимо, чтобы в грамматике было правило $A \rightarrow a_i$.

$$T_{i-1,i} = \{ A \mid A \rightarrow a_i \in R \}$$

Подстрока $a_{i+1} \dots a_j$ длины 2 и более может обладать свойством A , если она определяется некоторым правилом $A \rightarrow BC$. Тогда подстрока разбивается на две непустых подстроки, $a_{i+1} \dots a_k$ и $a_{k+1} \dots a_j$, причём первая должна обладать свойством B , а вторая — свойством C . Точка разбиения k может быть любым числом от $i + 1$ до $j - 1$. Так как обе эти подстроки *короче*, чем вся подстрока, то все сведения об этих более коротких строках к этому моменту уже вычислены и занесены в ячейки таблицы $T_{i,k}$ и $T_{k,j}$. Отсюда можно вычислить множество всех категорий для подстроки $a_{i+1} \dots a_j$.

$$T_{i,j} = \{ A \mid \exists A \rightarrow BC \in R, \exists k \in \{i + 1, \dots, j - 1\} : B \in T_{i,k} \wedge C \in T_{k,j} \}$$

Algorithm 2 Алгоритм Кокка–Касами–Янгера, см. Касами [1965], Янгер [1967]

Грамматика $G = (\Sigma, N, R, S)$ в н.в.Хомского, входная строка $w = a_1 \dots a_n$, где $n \geq 1$ и $a_i \in \Sigma$. Для всех $0 \leq i < j \leq n$, пусть $T_{i,j}$ — переменная, принимающая значение подмножества N .

```

1: for  $i = 1$  to  $n$  do
2:    $T_{i-1,i} = \{ A \mid A \rightarrow a_i \in R \}$ 
3: for  $\ell = 2$  to  $n$  do
4:   for  $i = 0$  to  $n - \ell$  do
5:     пусть  $j = i + \ell$ 
6:      $T_{i,j} = \emptyset$ 
7:     for all  $A \rightarrow BC \in R$  do
8:       for  $k = i + 1$  to  $j - 1$  do
9:         if  $B \in T_{i,k} \wedge C \in T_{k,j}$  then
10:           $T_{i,j} = T_{i,j} \cup \{A\}$ 
11: принять тогда и только тогда, когда  $S \in T_{0,n}$ 

```

Пример 15. Следующая грамматика в н.в.Хомского задаёт язык Дика без пустой строки.

$$\begin{aligned}
S &\rightarrow SS \mid AC \\
A &\rightarrow a \\
B &\rightarrow b \\
C &\rightarrow SB \mid b
\end{aligned}$$

Таблица разбора для строки $w = abaabb$ такова.

	0	1	2	3	4	5	6
0		{A}	{S}	$\emptyset$	$\emptyset$	$\emptyset$	{S}
1			{B, C}	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
2				{A}	$\emptyset$	$\emptyset$	{S}
3					{A}	{S}	{C}
4						{B, C}	$\emptyset$
5							{B, C}
6							

Поскольку $S \in T_{0,6}$, строка принадлежит языку.

6.5 Задачи распознавания свойств грамматик

Пустота: по данной грамматике G , определить, верно ли, что $L(G) = \emptyset$. Разрешима алгоритмом, очень похожим на алгоритм построения множества Nullable.

Однако очень похожая задача оказывается неразрешимой: «по данной грамматике G над алфавитом Σ , определить, верно ли, что $L(G) = \Sigma^*$ ».

Равносильность неразрешима. По данной МТ строится грамматика, задающая язык Σ^* тогда и только тогда, когда МТ не принимает никакую строку.

6.6 Нахождение наибольшей общей подпоследовательности

Пусть Σ — алфавит, а $a_1 \dots a_m \in \Sigma^*$ строка над этим алфавитом. Строка, полученная удалением 0 и более символов из строки, при сохранении относительного порядка оставшихся символов, называется её *подпоследовательностью*: то есть, если $1 \leq i_1 < \dots < i_k \leq m$, то строка $a_{i_1} \dots a_{i_k}$ — подпоследовательность строки $a_1 \dots a_m$.

Пусть $b_1 \dots b_n \in \Sigma^*$ — вторая строка. Требуется найти самую длинную общую подпоследовательность этих двух строк.

Перебирать все варианты — слишком долго, это будет экспоненциальное время относительно m и n .

Есть алгоритм, работающий за время $O(mn)$. Для всякого префикса (начального куска) $u_i = a_1 \dots a_i$ первой строки и для всякого префикса $v_j = b_1 \dots b_j$ второй строки алгоритм находит *длину* наибольшей общей подпоследовательности u_i и v_j и запоминает её в переменной $T_{i,j}$. Ячейки прямоугольной таблицы $T_{i,j}$ заполняются последовательно, начиная от более коротких префиксов, и заканчивая префиксами u_m и v_n — то есть, самими данными строками. Значение каждой ячейки $T_{i,j}$ вычисляется так. Если $i = 0$ или $j = 0$, то есть, один из двух префиксов пуст, то $T_{i,j} = 0$. Для всяких $i, j \geq 1$ рассматриваются два случая: если последние символы двух префиксов совпадают ($a_i = b_j$), то в наибольшей общей подстроке они соответствуют друг другу, а перед ними идёт наибольшая общая подстрока длины $T_{i-1,j-1}$ от более коротких префиксов; если же последние символы различны, то в наибольшей общей подстроке или a_i не соответствует никакому символу (и тогда остаётся подстрока длины $T_{i-1,j}$), или b_j не соответствует никакому символу (остаётся $T_{i,j-1}$). На основе этих соображений строится следующая формула.

$$T_{i,j} = \begin{cases} T_{i-1,j-1} & \text{если } a_i = b_j \\ \max(T_{i-1,j}, T_{i,j-1}) & \text{если } a_i \neq b_j \end{cases}$$

А Приложение: быстрое умножение матриц

Алгоритм быстрого умножения матриц Штрассена [1969] в целом похож на алгоритм быстрого умножения Карацубы.



Рис. 21: Фолькер Штрассен (род. 1936).

Пусть $\mathcal{R}$ — кольцо, т.е., есть ассоциативные операции сложения и умножения, сложение коммутативно, дистрибутивность, нейтральные элементы, противоположный элемент относительно сложения (без противоположного элемента — *полукольцо*).

Сумма матриц $A \in \mathcal{R}^{m \times n}$, $B \in \mathcal{R}^{m \times n}$ — матрица $A + B = C \in \mathcal{R}^{m \times n}$.

$$c_{i,j} = a_{i,j} + b_{i,j}$$

Сложение ассоциативно, коммутативно, есть нейтральный и противоположный элементы.

Произведение матриц $A \in \mathcal{R}^{m \times \ell}$, $B \in \mathcal{R}^{\ell \times n}$ — матрица $A \times B = C \in \mathcal{R}^{m \times n}$.

$$c_{i,j} = \sum_{k=1}^{\ell} a_{i,k} \cdot b_{k,j}$$

Умножение ассоциативно, есть нейтральный элемент.

Пример 16. Произведение двух матриц размера 2×2 .

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \times \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

Утверждение 1. Квадратные матрицы размера $n \times n$ над кольцом сами образуют кольцо.

Алгоритм для умножения матриц. Умножение двух матриц $n \times n$ по определению вычисляется за n^3 умножений и $n^2(n-1)$ сложений. Например, матрицы 2×2 умножаются за 8 умножений и 4 сложений (пример 16).

Лемма 4 (Штрассен [1969]). Пусть $\mathcal{R}$ — кольцо, пусть $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$ и $B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$ — две матрицы размера 2×2 над $\mathcal{R}$. Тогда их произведение $AB = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$ можно вычислить за 7 умножений и 18 сложений в $\mathcal{R}$.

Доказательство. Желаемые 8 произведений даны на рис. 22(left), где горизонтальные пунктирные линии соответствуют четырём элементам A , вертикальные — четырём элементам B , а пересечение соответствует произведению. Сплошные линии — суммы.

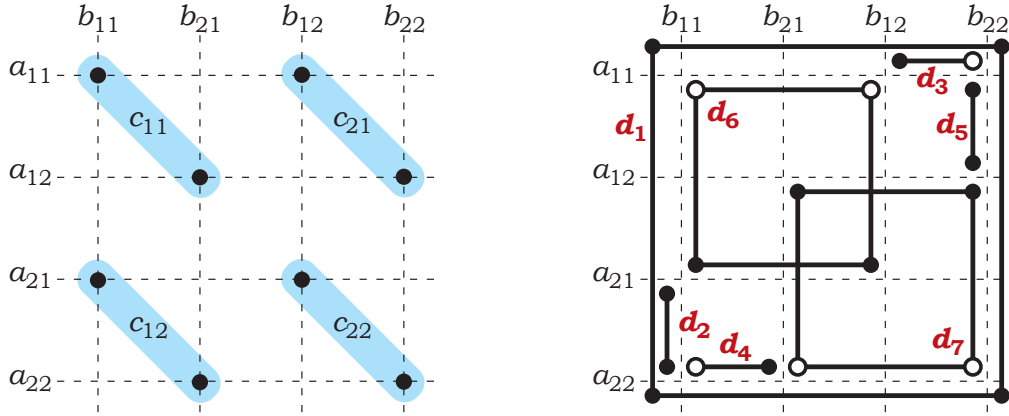


Рис. 22: (слева) Восемь произведений при умножении матриц 2×2 по определению; (справа) семь произведений при умножении по Штрассену.

Вычисляются следующие 7 произведений.

$$\begin{aligned}
 d_1 &= (a_{11} + a_{22})(b_{11} + b_{22}), \\
 d_2 &= (a_{21} + a_{22})b_{11}, \\
 d_3 &= a_{11}(b_{12} - b_{22}), \\
 d_4 &= a_{22}(b_{21} - b_{11}), \\
 d_5 &= (a_{11} + a_{12})b_{22}, \\
 d_6 &= (a_{21} - a_{11})(b_{11} + b_{12}), \\
 d_7 &= (a_{12} - a_{22})(b_{21} + b_{22}).
 \end{aligned}$$

Они изображены на рис. 22(right), где заполненный круг соответствует произведению со знаком «плюс», а пустой — произведению, умноженному на -1 . Тогда искомое произведение $C = AB$ выражается так.

$$\begin{aligned}
 c_{11} &= d_1 + d_4 + d_7 - d_5 \\
 c_{12} &= d_3 + d_5 \\
 c_{21} &= d_2 + d_4 \\
 c_{22} &= d_1 + d_3 + d_6 - d_2
 \end{aligned}$$

□

Этим же методом можно свести умножение матриц размера $n \times n$ к умножению 7 пар матриц размера $\frac{n}{2} \times \frac{n}{2}$ — ведь они же тоже образуют кольцо, и потому лемма применима.

Глубина рекурсии $\log_2 n$. Задач размера $n - 1$. Задач размера $\frac{n}{2} - 7$. Задач размера $\frac{n}{2^i} - 7^i$. Время работы для каждой задачи размера $\frac{n}{2^i} - O((\frac{n}{2^i})^2)$ шагов. Всего:

$$\sum_{i=0}^{\log_2 n} 7^i \frac{n^2}{2^{2i}} = n^2 \sum_{i=0}^{\log_2 n} \left(\frac{7}{4}\right)^i = n^2 \frac{\left(\frac{7}{4}\right)^{1+\log_2 n} - 1}{\frac{7}{4} - 1} = O(7^{\log_2 n}) = O(n^{\log_2 7})$$

Теорема 6 (Штрассен [1969]). Пусть $\mathcal{R}$ — кольцо, пусть $k \geq 0$, и пусть A и B — две матрицы $2^k \times 2^k$ над $\mathcal{R}$. Тогда произведение AB можно вычислить за 7^k умножений и $\Theta(7^k)$ сложений.

Отсюда матрицы размера $n \times n$ можно перемножить за время $O(n^{\log_2 7})$.

Список литературы

- [1959] N. Chomsky, “On certain formal properties of grammars”, *Information and Control*, 2:2 (1959), 137–167.
- [1936] A. Church, “An unsolvable problem of elementary number theory”, *American Journal of Mathematics*, 58 (1936), 345–363.
- [1988] N. Immerman, “Nondeterministic space is closed under complementation”, *SIAM Journal on Computing*, 17:5 (1988), 935–938.
- [1962] А. Карацуба, Ю. Офман, “Умножение многозначных чисел на автоматах”, *Доклады Академии Наук СССР*, 145:2 (1962), 293–294.
- [1965] T. Kasami, “An efficient recognition and syntax-analysis algorithm for context-free languages”, Report AF CRL-65-758, Air Force Cambridge Research Laboratory, USA, 1965.
- [1951] S. C. Kleene, “Representation of events in nerve nets and finite automata”, *RAND Research Memorandum* RM-704, 1951, 98 pp.
- [1963] О. Б. Лупанов, “О сравнении двух типов конечных источников”, *Проблемы кибернетики*, 9 (1963), 321–326.
- [1959] M. O. Rabin, D. Scott, “Finite automata and their decision problems”, *IBM Journal of Research and Development*, 3:2 (1959), 114–125.
- [1970] W. J. Savitch, “Relationships between nondeterministic and deterministic tape complexities”, *Journal of Computer and System Sciences*, 4:2 (1970), 177–192.
- [1969] V. Strassen, “Gaussian elimination is not optimal”, *Numerische Mathematik*, 13 (1969), 354–356.
- [1988] R. Szelepcsényi, “The method of forced enumeration for nondeterministic automata” *Acta Informatica*, 26:3 (1988), 279–284.
- [1937] A. M. Turing, “On computable numbers, with an application to the Entscheidungsproblem”, *Proceedings of the London Mathematical Society, Series 2*, 42:1 (1937), 230–265.
- [1967] D. H. Younger, “Recognition and parsing of context-free languages in time n^3 ”, *Information and Control*, 10 (1967), 189–208.